

RADICAL DENESTING / CODE REVIEW

A Second-Generation Audit of StradFixed2.wl

Correctness boundaries, search-state defects,
and a proposed StradFixed3.wl

5 September 2026

Principal conclusion. The package has a sound architectural idea—separate candidate generation from exact acceptance—and several correctly derived mathematical recipes. Its remaining problems are concentrated in input admission, the meaning of equality statuses, preservation of the incumbent, recursive search policy, and the scope of resource and reporting contracts. This review does not establish a concrete unequal numerical output on a valid, parameter-free algebraic input.

Evidence boundary. The complete 745-line source was reconstructed and its Git blob hash verified. Exact algebraic checks were executed in Python/SymPy. Native Wolfram execution was unavailable: the supplied revision and its 50 Wolfram regression tests are *not* represented as kernel-validated or production-ready.

Deliverable. The archive contains this article, standalone proposed code, the exact pinned original, a unified diff, a reproducible patch builder, independent verification scripts and results, native tests, and a source manifest.

Abstract

We audit the second corrected Wolfram Language implementation of **Strad**, taking the repository’s unified-B review as prior work rather than repeating its conclusions. Definite source-level findings include an overpermissive algebraicity predicate for opaque algebraic representations, a heuristic route to the supposedly exact status **"Different"**, two assignments that discard a better incumbent, and candidate guards that confuse an unchanged subproblem with an unhelpful outer transformation. We also identify gaps between stated and implemented resource boundaries, context-insensitive memoization, incomplete public argument validation, and ambiguous certificate telemetry. We distinguish these defects from intentional incompleteness and from correct code that should not be changed. A proposed revision retains the established proposal engine, strengthens its acceptance boundary, repairs search-state handling, and extends the cubic trace–norm method to higher odd indices using explicitly derived Dickson-polynomial identities. A multiquadratic example shows why completing only the visible square-class basis is insufficient. All computational claims are separated into independently executed exact checks, source deductions, and unexecuted native regression obligations.

Contents

1	Scope, provenance, and relation to earlier reviews	3
1.1	What was reviewed	3
1.2	What unified-B already established	3
1.3	Evidence labels used here	3
2	How the implementation works	4
2.1	The pipeline	4
2.2	What “improvement” actually means	4
2.3	A conditional correctness argument	4
3	Findings at a glance	5
4	Detailed review of the defects and boundaries	6
4.1	F01: the algebraicity admission test is not sound	6
4.2	F02: numerical rejection is not an exact inequality status	6
4.3	F03: the incumbent can be overwritten	7
4.4	F04: unchanged subproblems can still give useful outer candidates	8
4.5	F05: resource guarantees need a precise boundary	8
4.6	F06: memoization conflates searches with different resources	9
4.7	F07: malformed public arguments can miss the validator	9
4.8	F08–F09: progress counters and certificate records	9
4.9	F10–F12: limitations that should not be mislabeled as bugs	10
5	Mathematical audit: formulas that should be retained	11
5.1	Direct quadratic square-root denesting	11
5.2	Indirect denesting and the sign of the norm	11
5.3	The cubic trace–norm recipe is correct	11

5.4	Honsbeek-type proposals are identities plus branch checks	12
5.5	Principal phases and multiplier orbits	12
5.6	The Horner denominator inverse is correct	13
6	An implemented improvement: all higher odd indices in a real quadratic field	13
6.1	Finite formulas and an algebraic identity	13
6.2	A necessary and sufficient membership criterion	13
6.3	Special cases and implementation	14
7	A multi-surd obstruction to a too-small search space	14
8	What the proposed revision changes—and what it does not	15
8.1	Preserved structure	15
8.2	Compatibility changes that deserve attention	15
8.3	Remaining limitations	16
9	Verification and reproducibility	16
9.1	What actually ran	16
9.2	Native tests supplied but not executed	16
9.3	Rebuild commands	17
10	Prioritized next steps	17
11	Conclusion	18
A	Source map for future review	18
B	Minimal diagnostic probes	18
C	References and source qualifications	19

1 Scope, provenance, and relation to earlier reviews

1.1 What was reviewed

The principal object is `src/corrected/StradFixed2.wl` at the commit printed on the title page, not a moving copy of `main`. Its byte length is 41,416 and its Git blob identifier is

0bdeded347385a55a044fdd35883006d053685dc.

The included `code/StradFixed2.pinned.wl` reproduces that blob exactly. The patch builder checks the Git object hash before changing anything. All original source line references in this article refer to that file. The proposed package uses a different public context, `RadicalDenest3`, so it can coexist with `RadicalDenest2` during differential testing. No repository files were modified remotely.[1]

The contextual reading covered the repository’s unified denesting report, unified-B’s design and experimental discussion, the relevant direct/indirect square-root and quadratic-field sections, and literature on field-based denesting and implementation. The repository’s re-typeset literature collection contains substantive editorial corrections, not merely facsimiles. For example, the BFHT corrections file explicitly qualifies recursive witness bookkeeping, and the edited Landau extended abstract distinguishes the base-field and roots-of-unity hypotheses. These editorial versions are useful research aids, but are not silently substituted for the original authors’ claims.[3, 2, 4, 5, 6]

The original BFHT paper, Gkioulekas’s author manuscript, and Jeffrey–Rich’s chapter were also inspected in their public PDF forms, including page images. Cavallo’s cubic-field criterion and Wolfram’s current documentation supplied additional checks. The bibliography distinguishes an original publication, an author manuscript, repository editorial material, and implementation documentation.[7, 9, 10, 11, 13]

1.2 What unified-B already established

Unified-B describes a substantial improvement over the first corrected implementation: a centralized gate, principal-branch handling, whole-island proposals, bounded search, explicit option validation, and additional shape-specific recipes. It reports 116 benchmark rows, 340 structured randomized inputs, and a 144-test regression suite; those are *upstream-reported* experiments, not new measurements in this audit. Its report says the benchmark and randomized outputs were exactly equal to their inputs and that the fifteen previously listed gaps were resolved.[2]

Those results are important positive evidence. They do not establish that every opaque `Root` accepted by a predicate is algebraic, that every helper preserves the best proposal, or that every operation obeys the same resource contract. These are different properties from successfully denesting the existing example families. In particular, unified-B already acknowledged the heuristic nature of numerical rejection and incomplete multi-surd coverage. This review classifies those as retained limitations or unresolved contract issues rather than newly discovered numerical failures.

1.3 Evidence labels used here

A *source finding* is a statement directly implied by the pinned code’s control flow or data validation. A *mathematical finding* is supported by an explicit derivation, sometimes also by independently executed exact arithmetic. A *native probe* is Wolfram code supplied to confirm

evaluator-sensitive behavior; none of the new native probes was executed in this session. A *design limitation* is a deliberate scope restriction or a plausible improvement opportunity, not automatically a bug.

There was no local Wolfram kernel, and an actual attempt to use the available Wolfram service failed because its endpoint was unavailable. Consequently this article makes no claim about measured runtime improvement, no claim that the revised package passes native tests, and no claim to have reproduced an unequal output on an admitted, genuinely algebraic, parameter-free input.

2 How the implementation works

2.1 The pipeline

The public entry points resolve effective options, establish a session, and traverse supported hosts. An exact algebraic island is passed to `improveNumber`. Rational-power islands first receive specialized proposals, negative-radical phase separation, and Kummer-style factorization/index reduction. Other proposals come from `RootReduce`, `ToRadicals`, `Simplify`, `Factor`, a polynomial denominator inverse, and an optional user solver. A bounded multiplier search is then available for suitable radical nodes and products. Rewritten components can also be recombined as a whole island. *Pinned source, lines 119–143, 578–727.*

This is an important improvement over a purely local replacement rule. A sum can cancel globally even when none of its summands has a depth-one real-radical representation. Conversely, a useful nested-power transformation can require a temporary representation that is not itself the cheapest expression. The repository report discusses both phenomena.[3]

The central acceptance rule requires a candidate to be an explicit radical expression, satisfy the leaf-count guard, be strictly cheaper than the incumbent, and pass an equality check against the target of that island. *Pinned source, lines 223–249.* A callback is therefore a proposal generator, not an authority. This architecture should be retained.

2.2 What “improvement” actually means

The cost is a lexicographically ordered five-tuple:

$$C(E) = (o(E), d(E), r(E), \ell(E), b(E)),$$

where the components count opaque representations, radical depth, rational-power nodes, leaves, and integer/rational bit size. An opaque `Root` counts as depth zero but is penalized in the *first* component. Thus converting an opaque algebraic number into a radical expression may increase the displayed radical depth while still decreasing the declared cost. That is consistent with the ordering; it is not a proof that radical depth decreased. *Pinned source, lines 167–183.*

The comparison `Order[RadicalCost[new], RadicalCost[old]] == 1` has the correct direction for these equal-length integer tuples. It should not be reversed. More generally, equality, radical presentation, syntactic depth, algebraic degree, and cost minimization are different tasks. For example, $\sqrt{5 + 2\sqrt{6}} = \sqrt{2} + \sqrt{3}$ has degree four over $\mathbb{Q}$ on both sides, but the displayed nesting depth changes from two to one.[3, 7]

2.3 A conditional correctness argument

Proposition 2.1 (Acceptance and congruence). *Assume the input is an evaluated, pure arithmetic expression; every admitted numeric island is genuinely algebraic; the exact kernel*

operations used to certify equality are correct; and a changed island is accepted only after an exact equality certificate. Then replacing those islands through arithmetic constructors preserves the value wherever the original expression is defined. If every committed pass is also strictly cheaper, the returned expression either equals the original representation or has lower declared cost.

Proof. For each replacement, the certificate states equality of the old and new complex numbers, including their selected branches. Applying the same arithmetic constructor to equal arguments preserves equality whenever that constructor is defined. Induction over the traversed expression gives equality of the complete rebuilt expression. The final pass check discards a non-improving rebuild, and a whole-number input receives an additional whole-input certificate. Transitivity proves equality after several passes. Strict cost descent is enforced only at the commit points, which proves the stated cost property. $\square$

The assumptions matter. A numerical nonzero hint is not an exact inequality proof. A syntactically exact `Root` is not necessarily algebraic. Arbitrary side-effecting upvalues need not respect mathematical congruence. The remainder of the review examines these boundaries rather than treating the gate’s existence as a complete proof of the package’s contract.

3 Findings at a glance

ID	Priority / type	Finding	Disposition in proposal
F01	High; input	Exact opaque objects need not be algebraic.	Validate polynomial and generator data.
F02	High; API	A numerical hint can produce the exact-looking status <code>"Different"</code> .	Hints never decide; default off.
F03	Medium; search	Two later stages can discard a cheaper incumbent.	Preserve the incumbent.
F04	Medium; search	An unchanged subproblem can still improve the outer target.	Judge the outer candidate.
F05	Medium; limits	Some work and report costs escape advertised resource regions.	Add wrappers; budget costs.
F06	Medium; cache	Reduced-budget failures can suppress later searches.	Context keys; no negative cache.
F07	Medium; API	Malformed arguments can evade public pattern matching.	Validate raw arguments.
F08	Low; counts	Fast-path success can be counted without acceptance.	Compare before and after.
F09	Medium; evidence	Truncated proposal records are not independent proof objects.	Label scope and truncation.
F10	Known limit	Multi-surd ansatz misses square classes and scalar cosets.	Prove the scope limitation.
F11	Conditional	Opaque payload scans can overlap; Gaussian size is uncharged.	Structural traversal and cost.
F12	Scope	Host policies, local cost, and checkpoint rollback limit guarantees.	Clarify remaining limits.

4 Detailed review of the defects and boundaries

4.1 F01: the algebraicity admission test is not sound

Pinned source, lines 150–165, 189–211.. The relevant branch is essentially

```
opaqueQ[e], Precision[e] === Infinity && FreeQ[e, _Real]
```

with `opaqueQ` matching either `Root` or `AlgebraicNumber`. Wolfram documents `Root` representations with symbolic parameters and with exact transcendental coefficients. Infinite precision describes exactness of representation; it does not establish algebraicity over $\mathbb{Q}$.^[13]

A useful probe is

```
z = Root[#^5 + # - Pi &, 1];
ExactAlgebraicQ[z]
```

Every value of this root is transcendental. Indeed, if z were algebraic, then $z^5 + z$ would be algebraic, whereas the defining equation says $z^5 + z = \pi$. This is a mathematical counterexample to the *predicate's claimed criterion*; its concrete native evaluation is supplied as a regression obligation, not reported as an executed test. A parameterized root such as `Root[##^5 + ## + a &, 1]` exposes the separate problem that exact symbolic objects need not be parameter-free numbers.

The problem propagates into certification. `canonicalNonzeroQ` also trusts exact opaque heads, and the fallback `PossibleZeroQ[... , Method -> "ExactAlgebraics"]` has a documented exact guarantee for *explicit algebraic numbers*. The admission test is supposed to establish that precondition. A false positive takes execution outside the justification for the gate. This identifies a real trust-boundary gap, but does not by itself prove that a particular algebraically unequal candidate was accepted.^[14]

The proposed repair admits a `Root` only after its defining pure function yields a nonconstant polynomial with rational coefficients and a valid integer root selector; the supported optional root-isolation selector is also checked. `AlgebraicNumber` requires an admitted generator and a rational coefficient list. The construction is conservative: algebraic-coefficient or triangular `Root` presentations may be rejected even when mathematically algebraic. A caller can separately normalize a *known* algebraic object with `RootReduce`. This is an explicit compatibility change, not an unnoticed loss of scope.

The nonzero shortcut is correspondingly tightened. A validated rational polynomial with nonzero constant term has no zero root. More complicated representations are left to the exact zero test instead of being declared nonzero from their heads. An eventual broader implementation should return a structured admission result—valid, invalid, unsupported, or resource-limited—rather than forcing all four outcomes into a Boolean.

4.2 F02: numerical rejection is not an exact inequality status

Pinned source, lines 192–211.. The prefilter numerically approximates a difference at 40 digits and requires reported accuracy at least 25 and magnitude greater than 10^{-20} . On success, `certify` immediately returns "Different". That is stronger than “this candidate looks unpromising.”

Significance arithmetic and `Accuracy` metadata are useful diagnostics, but the code does not construct a rigorously outward-rounded enclosure excluding zero. The fixed threshold is not a proved root-separation bound either. Hence this branch does not meet the stated

exact three-valued API contract. This retained issue was already discussed in unified-B; the disagreement is about the semantics of the public status, not about whether heuristic screening can be useful.[15, 2]

A subtle distinction prevents overstatement. For genuinely algebraic inputs, this heuristic is rejection-only: it cannot on its own certify a wrong equality. Its direct risks are false negative denesting results and false assertions of inequality. F01 is more serious for the acceptance boundary because it undermines the domain hypothesis of the subsequent exact method.

The revision disables the numerical prefilter by default. When explicitly enabled, the old calculation records a `NumericHints` event but proceeds to exact certification. `NumericRejections` is retained as a compatibility counter and is no longer incremented. A test deliberately injects a false numerical hint for the classical identity $\sqrt{5 + 2\sqrt{6}} = \sqrt{2} + \sqrt{3}$; the revised exact route must still return "Equal". That injection tests the decision boundary. It is *not* offered as an observed failure of Wolfram's numerical evaluation on that identity.

A future rigorously certified interval filter could safely return "Different" when its enclosure excludes zero. Until then, the honest alternatives are a scheduling hint or "Unknown", not a theorem-like negative answer.

4.3 F03: the incumbent can be overwritten

Pinned source, lines 435–461, 600–619.. `powerProposals` starts with an incumbent and offers fast-path candidates against it. However, its later assignments are

```
best = negativeRadicandCandidate[target, rho, p, q];
...
best = kummerCandidates[target, rho, p, q];
```

The negative helper returns `target` for a nonnegative radicand, and the Kummer helper initializes its own `best` to `target`. Neither receives the earlier incumbent. Consequently the acceptance gate can be perfectly correct while the proposal pipeline is non-monotone: a later stage can replace a better expression by the original one.

The early exit when depth is at most one hides this problem in many familiar examples. It does not protect a cheaper depth-two result, or a partial denesting from depth three to depth two. The defect is therefore a loss of search progress, not automatically a wrong mathematical value.

The simplest repair is to make the contract of every proposal stage explicit:

$$\text{stage}(E, B) = B' \implies B' = B \text{ or } (B' = E \text{ in value and } C(B') < C(B)).$$

Here the equality phrase concerns values, not syntax. In the proposed code, both helpers take the incumbent and initialize from it. Their caller passes `best`. This is preferable to blindly assigning a helper result and then trying to reconstruct lost state.

For regression isolation, use

$$E = (3 + 2\sqrt{2})^{1/4}, \quad B = \sqrt{1 + \sqrt{2}}.$$

Both denote the same positive number, both have depth two, and B is a shorter presentation. The supplied native test replaces expensive factor-search helpers by deterministic stubs and checks that the stage does not lose B . A mocked control-flow test is appropriate here precisely because the default downstream search might rediscover the expression and conceal the defect.

4.4 F04: unchanged subproblems can still give useful outer candidates

Pinned source, lines 445–451, 613–619.. Index reduction contains the guard

```
If[sub != den^(1/rest) &&
  RadicalDepth[sub] < RadicalDepth[target], ...]
```

The first condition asks whether a recursive call changed its own input. That is not the relevant question. A field factor may already have transformed the original problem into a cheaper subproblem, and the recursive call may correctly return that subproblem unchanged.

In the preceding fourth-root example, the field root $\gamma = 1 + \sqrt{2}$ satisfies $\gamma^2 = 3 + 2\sqrt{2}$. The unchanged residual expression $\sqrt{\gamma}$ is already the useful outer proposal. Moreover, the strict depth comparison rejects improvements that preserve depth but improve a later component of the declared cost. Both restrictions are stronger than the package's actual acceptance policy.

The revision admits an exact residual candidate of depth no greater than the target, then delegates equality and strict cost descent to the common gate. The even-index split receives the same treatment. The negative-radical helper also offers a phase-separated proposal rather than requiring the modulus subproblem to have changed first. These edits enlarge candidate coverage; they do not claim that every affected example failed through every original fallback.

4.5 F05: resource guarantees need a precise boundary

Pinned source, lines 92–111, 350–373, 420–427, 560–571, 692–727.. The header's claim that every expensive kernel operation has its own bounded region is not literally true. Counterexamples include **FactorInteger** when collecting multi-surd primes, expansion of the symbolic square in that solver, discriminant factorization in multiplier generation, and **Together** used while extracting linear-factor roots.

These operations are generally still inside the *outer* session constraint. It would therefore be inaccurate to call the entire search unbounded. The defect is that an operation can consume substantially more than "**OperationTime**" and interfere with the intended allocation of work. The patch adds local wrappers and handles the resulting failure sentinels conservatively.

There is a second, distinct boundary issue. Original input classification and the final **InitialCost** and **FinalCost** calculations occur outside the principal **TimeConstrained** and **MemoryConstrained** body. Thus even a zero search budget can be followed by a large report traversal. The revision performs input classification and cost construction inside the search region, stores the cost alongside the committed expression, and returns **Missing["NotComputed"]** when a disabled or interrupted session has not computed it.

Ordinary Wolfram argument evaluation, evaluation of option expressions, and final assembly of the report still lie outside that region. Eliminating those effects would require a different held-input API with carefully specified evaluation semantics, not just one more timeout wrapper. Nor is an in-kernel constraint an operating-system sandbox. Wolfram documents interrupt granularity, interaction with **AbortProtect**, and process-accounting qualifications for **TimeConstrained**.^[16]

Finally, **MaxDegree** checks a polynomial *after* its construction. It is not a bound on the cost of computing that polynomial or on the degree of a compositum used for factorization. **MaxLeafCount** is not a peak-intermediate-size limit. **MaxTrials** limits a multiplier-search visit, not all recursively related work throughout a session. The revised documentation states these narrower meanings. Hard wall-clock and process-memory enforcement would require a supervised worker kernel.

4.6 F06: memoization conflates searches with different resources

Pinned source, lines 430–433, 626–640.. Recursive calls reduce `MaxTrials` by a factor of four and increase recursion depth. Nevertheless, the original cache key is just the target expression, and an unchanged result is cached whenever the overall deadline has not expired. A search that reached a local limit can therefore suppress a later search of the same syntax with more available trials or recursion depth.

This does not make a returned equality false. It makes the cache behave as though “no improvement found under this context” meant “no further improvement available.” The distinction is especially important because operation timeouts can occur before the global deadline and because the same expression can reappear through index reduction, roots-of-unity orbits, and repeated passes.

The revision keys a cache entry by the expression, effective trial limit, and remaining recursion allowance; stores only beneficial results; and tracks expressions active on the current recursive path. The active-path test prevents rediscovering the same recursive problem until the depth cap intervenes. A positive cache entry is still a heuristic reuse of a certified candidate, not a proof that the candidate is optimal. Wall time, kernel cache state, and proposal ordering can affect what a later search might discover. A complete search-status cache is deferred rather than claimed solved.

4.7 F07: malformed public arguments can miss the validator

Pinned source, lines 119–143, 736–742.. The option resolver correctly rejects non-rule entries *if it is called*. Public definitions using `opts : OptionsPattern[]` need not match a call such as

```
Strad[expr, 17]
```

so the validator’s non-rule branch does not itself guarantee the advertised behavior for every malformed invocation. The result can remain an unevaluated public call rather than a structured `Failure`.

The revision accepts raw trailing arguments, recognizes the optional positional Boolean for the applicable entry points, and then calls the existing rule validator. Missing-input cases receive explicit failures. Effective defaults remain those of the entry point actually invoked, and first-explicit-rule precedence is preserved. The change intentionally turns formerly unevaluated malformed calls into failures; this should be treated as an API correction and tested, not hidden inside a mathematical rewrite.

4.8 F08–F09: progress counters and certificate records

Pinned source, lines 76–86, 226–234, 606–609, 723–727.. The original `FastPathAccepted` counter increments when the current result has depth at most one, even if no fast-path candidate was accepted. A rational power over an opaque root is one reason a depth test alone is not an acceptance event. The revision compares the incumbent before and after the fast-path stage instead.

The array named `Certificates` records `Before`, `After`, and `Method` for accepted proposals, up to `MaxTraceEntries`. These are valuable audit records, but they are neither independently checkable algebraic proof objects nor necessarily a derivation of the final committed output. A proposal may later be superseded, discarded by F03, or rolled back with an interrupted or globally non-improving pass. Recursive subproblems also contribute records.

The patch labels the records as kernel-checked accepted proposals, adds an equality-status

field and a scope field, and exposes truncation. It does not pretend that these labels create a proof export system. A future certificate format should identify the input occurrence, algebraic-number embedding, exact verification method, and commit lineage. Polynomial relations alone are insufficient: two distinct conjugates can satisfy the same polynomial.

A limit flag likewise must not be read as a proof that a bound obstructed a specific denesting. Some original flags mean a cap was reached, even if the currently available queue happened to end at the same point. The report now states that these are triggered guards or reached caps, not non-denestability certificates.

4.9 F10–F12: limitations that should not be mislabeled as bugs

The two multi-surd trial bases are intentionally incomplete; Section 7 gives a precise example and a stronger warning about visible-prime closure. The finite multiplier queue is also heuristic. For $\rho = 2 + \sqrt{3}$, the quadratic norm is one, yet multiplying by two enables

$$\sqrt{\rho} = \frac{\sqrt{6} + \sqrt{2}}{2}.$$

Thus norm divisors alone do not enumerate all useful multipliers. This observation already appears in unified-B.[2]

The original policy leaves any tree containing an inexact number unchanged, and traversal is restricted to designated arithmetic/list constructors. Held expressions and unknown heads are opaque. Those are conservative choices, not missing algebraic identities. A future opt-in traversal policy could process exact islands inside mixed-precision hosts, but it must specify whether reconstruction is allowed to trigger numerical reevaluation. *Pinned source, lines 678–707.*

The cost implementation subtracts counts over every opaque payload from a whole-tree count. In representations with overlapping opaque payloads this can subtract the same inner radical more than once. No fully evaluated native negative-count witness is claimed here; the concern is structural and depends on the representation admitted. The patch uses recursive stopping at the first opaque object instead. It also charges the integer/rational components of exact Gaussian constants. The latter is a cost-policy improvement, not an arithmetic correctness fix, and can change tie-breaking among equal expressions.

Local improvements are not necessarily global improvements. If another branch already determines the maximum depth, lowering one subtree’s depth while increasing its node count may increase the whole expression’s cost. The final rollback is then correct but may discard other beneficial local replacements made in the same pass. A Pareto frontier or per-occurrence transactional optimizer can address this; neither minimum-depth nor minimum-cost optimization is claimed here.

Finally, congruence is a mathematical property of pure operations, not a sandbox for arbitrary Wolfram programs. A symbol’s upvalues can deliberately distinguish two syntactic presentations of the same number or cause side effects during reconstruction. Such behavior must be outside the contract. Likewise, on interruption the package retains a completed checkpoint, not necessarily the best candidate ever generated during the interrupted pass.

5 Mathematical audit: formulas that should be retained

5.1 Direct quadratic square-root denesting

Let $a, b, c \in \mathbb{Q}$, $c > 0$ nonsquare, $b \neq 0$, and $\rho = a + b\sqrt{c} > 0$. Put $D = a^2 - b^2c$. If $D = s^2$ with $s \in \mathbb{Q}_{\geq 0}$ and $a > 0$, then

$$\sqrt{\rho} = \sqrt{\frac{a+s}{2}} + \operatorname{sgn}(b)\sqrt{\frac{a-s}{2}}. \quad (1)$$

Indeed, $a > s \geq 0$ because $a^2 - s^2 = b^2c > 0$. The square of the right side is $a + b\sqrt{c}$, and the selected sum or difference is positive. This verifies both the identity and the branch. The formula and its standard direct-denesting criterion are discussed in the repository report and the square-root literature.[3, 9]

There is also a useful necessity argument for denesting with square roots of positive rationals. Suppose $\alpha = \sqrt{\rho}$ lies in a real multiquadratic field M . Let H fix $\mathbb{Q}(\sqrt{c})$ and choose σ sending $\sqrt{c}$ to $-\sqrt{c}$. For $h \in H$, $h(\alpha) = \pm\alpha$. Since the Galois group is abelian of exponent two, $\alpha\sigma(\alpha)$ is fixed by H and by σ , hence is rational. Squaring it gives D . This explains why the norm test is an obstruction, not merely a convenient guess.

5.2 Indirect denesting and the sign of the norm

Suppose $D < 0$ and

$$e = \sqrt{b^2 - a^2/c} \in \mathbb{Q}.$$

Positivity of ρ then forces $b > 0$, and $0 < e \leq b$. The correct formula is

$$\sqrt{\rho} = c^{1/4} \left(\sqrt{\frac{b+e}{2}} + \operatorname{sgn}(a)\sqrt{\frac{b-e}{2}} \right). \quad (2)$$

The square of the bracket is $b + a/\sqrt{c}$; multiplication by $\sqrt{c}$ gives ρ . The bracket is positive, including the case $a = 0$. Moreover,

$$-cD = c^2e^2,$$

so the rational-square criterion involves $-cD$, not cD . This sign is already correct in StradFixed2. For example,

$$\sqrt{4 + 3\sqrt{2}} = 2^{1/4}(1 + \sqrt{2}).$$

The fourth-root extension is essential here if one asks for a depth-one positive-real radical representation. The real/complex scope of stronger classification theorems must be respected.[3, 7, 9]

5.3 The cubic trace–norm recipe is correct

For a real cube root $\beta = A + B\sqrt{c}$ with rational A, B , define

$$n = A^2 - cB^2, \quad T = 2A.$$

If $\beta^3 = a + b\sqrt{c}$, then

$$n^3 = a^2 - b^2c, \quad T^3 - 3nT = 2a, \quad B = \frac{b}{T^2 - n}. \quad (3)$$

Conversely, rational n, T satisfying the first two equations give the stated reconstruction. The identity

$$(T^2 - 4n)(T^2 - n)^2 = 4b^2c$$

shows that the denominator is nonzero and that $cB^2 = T^2/4 - n$. Expanding $(T/2 + B\sqrt{c})^3$ then proves the result. A rational cube norm by itself is not sufficient; the rational trace condition is also required. This is the package's normalized form of the quadratic-field cubic criterion.[2, 11]

Cavallo's paper uses a differently scaled trace variable. The two parameterizations agree after substituting the rational cube root of the norm. They should not be compared by identifying differently normalized symbols. Nor should real odd-root formulas be substituted directly for principal complex powers on negative inputs.

5.4 Honsbeek-type proposals are identities plus branch checks

Let $A^3 = \alpha$ and $B^3 = \beta$ be nonzero rationals, with A, B real and $A + B > 0$. Set $q = \beta/\alpha$. If a rational s satisfies

$$s^4 + 4s^3 + 8qs - 4q = 0,$$

then

$$\left(-\frac{s^2}{2}A^2 + sAB + B^2\right)^2 = (\beta - s^3\alpha)(A + B).$$

This follows by expanding

$$\left(-s^2/2 + su + u^2\right)^2 - (u^3 - s^3)(1 + u) = \frac{s^4 + 4s^3 + 8u^3s - 4u^3}{4}$$

and setting $u = B/A$. When $\beta - s^3\alpha > 0$, the two signs of the resulting quotient are valid candidate branches, and the exact gate selects the desired square root. This is the identity underlying the existing implementation; no unrestricted completeness claim follows from enumerating its rational parameters.[3, 2, 12]

For $A = \sqrt[3]{5}$, $B = -\sqrt[3]{4}$, the parameter $s = -2$ gives denominator square 36, leading to the familiar positive identity

$$\sqrt{\sqrt[3]{5} - \sqrt[3]{4}} = \frac{\sqrt[3]{2} + \sqrt[3]{20} - \sqrt[3]{25}}{3}.$$

The independent verification script checks the polynomial identity and these rational parameters exactly.

5.5 Principal phases and multiplier orbits

For negative real ρ and a reduced rational exponent p/q , the principal convention gives

$$\rho^{p/q} = (-1)^{p/q}(-\rho)^{p/q}. \quad (4)$$

This follows from $\text{Arg } \rho = \pi$ and the real logarithm of $-\rho > 0$. It does not equate a negative real odd root with a principal complex power.

For $k \mid q$, write $r = q/k$. If $\gamma^k = \rho$, the family

$$\left(\gamma\zeta_k^j\right)^{1/r} \zeta_r^\ell, \quad 0 \leq j < k, \quad 0 \leq \ell < r, \quad (5)$$

contains every q th root of ρ . To see this, the r th power of any such root is a k th root of ρ , hence equals one $\gamma\zeta_k^j$; the remaining factor accounts for its r possible roots. Raising these candidates to the integer power p and certifying against the target selects the correct rational power.

Similarly, if $z^q = m\rho$ and $m \neq 0$, then $(z/m^{1/q})^q = \rho$, but the quotient need not be the *principal* root. The multiplier algorithm's roots-of-unity orbit is therefore mathematically justified and should not be removed. The independent tests verify phase coverage using rational multiples of π , including points on the negative-real cut, without floating-point arithmetic.

5.6 The Horner denominator inverse is correct

Let $d \neq 0$ be algebraic and let

$$p(X) = a_0 + a_1X + \cdots + a_nX^n, \quad p(d) = 0, \quad a_0 \neq 0.$$

Then

$$\frac{1}{d} = -\frac{a_1 + a_2d + \cdots + a_nd^{n-1}}{a_0}. \quad (6)$$

The original `Fold` constructs precisely this polynomial in Horner form, including the degree-one case. The additional check of d times the proposed inverse against one is appropriate. There is no sign or coefficient-order bug here. The inverse may be algebraically valid while increasing the declared denesting cost; accordingly the public rationalization helper and the cost-gated proposal have different purposes. *Pinned source, lines 256–278.*

6 An implemented improvement: all higher odd indices in a real quadratic field

The repository report proposes trace-polynomial methods beyond the cubic case but leaves reconstruction and verification to be supplied.[3] The following derivation gives a uniform recipe. It is an application of classical trace/Dickson identities, not a claim to have discovered those polynomial families.

6.1 Finite formulas and an algebraic identity

For $m \geq 1$, define

$$D_m(T, n) = \sum_{j=0}^{\lfloor m/2 \rfloor} \frac{m}{m-j} \binom{m-j}{j} (-n)^j T^{m-2j}, \quad (7)$$

$$U_{m-1}(T, n) = \sum_{j=0}^{\lfloor (m-1)/2 \rfloor} \binom{m-1-j}{j} (-n)^j T^{m-1-2j}. \quad (8)$$

If $u + v = T$ and $uv = n$, direct multiplication or the recurrence for powers yields

$$D_m(T, n) = u^m + v^m, \quad (u - v)U_{m-1}(T, n) = u^m - v^m.$$

These identities also follow inductively from the common recurrence $P_{j+1} = TP_j - nP_{j-1}$ and the respective initial values. Therefore

$$D_m(T, n)^2 - 4n^m = (T^2 - 4n)U_{m-1}(T, n)^2. \quad (9)$$

This is a polynomial identity, so it remains valid when $u = v$ as well.

6.2 A necessary and sufficient membership criterion

Theorem 6.1 (Odd root in a specified real quadratic field). *Let $m \geq 3$ be odd, let $c > 0$ be a nonsquare rational, and let $a, b \in \mathbb{Q}$ with $b \neq 0$. The real m th root of $a + b\sqrt{c}$ belongs to $\mathbb{Q}(\sqrt{c})$ if and only if there exist $n, T \in \mathbb{Q}$ such that*

$$n^m = a^2 - b^2c, \quad D_m(T, n) = 2a. \quad (10)$$

When these conditions hold, $U = U_{m-1}(T, n)$ is nonzero and the root is

$$\beta = \frac{T}{2} + \frac{b}{U}\sqrt{c}. \quad (11)$$

Proof. For necessity, write $\beta = A + B\sqrt{c}$ and let $\gamma = A - B\sqrt{c}$ be its conjugate. Then $n = \beta\gamma$ and $T = \beta + \gamma$ are rational. Since $\beta^m = a + b\sqrt{c}$ and $\gamma^m = a - b\sqrt{c}$, taking products and sums gives (10).

For sufficiency, substitute (10) into (9):

$$(T^2 - 4n)U^2 = 4b^2c > 0.$$

Thus $U \neq 0$ and $T^2 - 4n > 0$. Put $A = T/2$, $B = b/U$, $\beta = A + B\sqrt{c}$, and $\gamma = A - B\sqrt{c}$. The displayed identity gives $cB^2 = A^2 - n$, hence $\beta + \gamma = T$ and $\beta\gamma = n$. Consequently

$$\beta^m + \gamma^m = 2a, \quad \beta^m - \gamma^m = (\beta - \gamma)U = 2b\sqrt{c}.$$

Adding the equations proves $\beta^m = a + b\sqrt{c}$. All quantities $A, B, \sqrt{c}$ are real; since m is odd, a real m th root is unique. This selects the required root. $\square$

Corollary 6.2. *Under these hypotheses there is at most one rational trace T satisfying (10) for the uniquely determined real norm root n . Every such trace reconstructs the same real root and therefore has the same rational component $T/2$.*

The theorem is a criterion for membership in *this quadratic field*, not for all possible depth-one denestings using arbitrary radicals. A failure of the rational trace test must not be reported as universal non-denestability.

6.3 Special cases and implementation

For $m = 3$, $D_3 = T^3 - 3nT$ and $U_2 = T^2 - n$, exactly recovering the original cubic code. For $m = 5$,

$$D_5 = T^5 - 5nT^3 + 5n^2T, \quad U_4 = T^4 - 3nT^2 + n^2.$$

With $n = -1$, $T = 2$, $a = 41$, and $b = -29$, formula (11) yields the real root $1 - \sqrt{2}$. The principal fifth power of $41 - 29\sqrt{2} < 0$ instead requires the phase in (4).

The new `quadraticOddRoots` helper computes the real norm root by an exact rational-power test, factors $D_m(T, n) - 2a$ over $\mathbb{Q}$ to obtain rational roots, reconstructs the candidate with (11), and verifies its m th power exactly. It handles odd indices at least five up to `MaxRootIndex`; the established cubic helper is retained. The outer common gate still certifies the principal target and cost reduction. Finite sums avoid auxiliary symbolic recurrence state, and expensive generation/factorization is bounded.

This addition can avoid a more general extension-field factorization on suitable inputs. That is an algorithmic opportunity, not a measured speedup. Depending on the input and kernel, factoring the trace polynomial may cost more than another route. No native performance comparison was available.

7 A multi-surd obstruction to a too-small search space

Pinned source, lines 350–373. The current solver extracts visible radicand primes P and tries only two supports, essentially $\{1\} \cup P$ and $\{1\} \cup P \cup \{d_i\}$. Completing the square-class span is a natural improvement, but even that does not solve every problem.

Proposition 7.1 (An external scalar square class). *Set*

$$\beta = \sqrt{30} + \sqrt{42} + \sqrt{70}, \quad \rho = 142 + 28\sqrt{15} + 20\sqrt{21} + 12\sqrt{35}.$$

Then $\sqrt{\rho} = \beta$, but $\beta \notin \mathbb{Q}(\sqrt{3}, \sqrt{5}, \sqrt{7})$. Thus no rational-coefficient square-root ansatz using only square classes generated by the visible primes 3, 5, 7 can represent the desired root.

Proof. Squaring β gives the rational term $30 + 42 + 70 = 142$ and cross terms

$$2\sqrt{30 \cdot 42} = 12\sqrt{35}, \quad 2\sqrt{30 \cdot 70} = 20\sqrt{21}, \quad 2\sqrt{42 \cdot 70} = 28\sqrt{15}.$$

All summands of β are positive, so β is the principal square root. In $\mathbb{Q}(\sqrt{2}, \sqrt{3}, \sqrt{5}, \sqrt{7})$, the automorphism that changes the sign of $\sqrt{2}$ and fixes the other three generators fixes the proposed visible-prime field and sends β to $-\beta$. Since $\beta \neq 0$, it cannot belong to that fixed field. $\square$

This is a provable miss of the *restricted multi-surd ansatz*, not a claim that the original full **Strad** call returns unchanged. The existing multiplier fallback may recover it. In fact,

$$\sqrt{2\rho} = 2(\sqrt{15} + \sqrt{21} + \sqrt{35}),$$

so multiplier two moves the problem into the visible field. The independent script checks both expansions using exact squarefree-basis arithmetic.

The example suggests two distinct upgrades. First, use square classes as vectors over $\mathbf{F}_2$ and work with their rank rather than the raw number of primes; this removes redundant generators. Second, search a controlled family of scalar cosets $\sqrt{m}K$ as well as K itself. A completed basis *inside* K does not remove the need to enlarge the field. Any finite list of scalar cosets must still be labeled heuristic unless accompanied by a proved sufficiency theorem.

8 What the proposed revision changes—and what it does not

8.1 Preserved structure

The revision retains the original direct and indirect quadratic formulas, cubic helper, Gaussian and Honsbeek recipes, multiplier search, exact acceptance gate, branch orbits, denominator inverse, and conservative supported-host traversal. It is not a wholesale replacement by **ToRadicals** or **FullSimplify**. The original source is included for comparison; `build_improved.py` checks its hash and applies 36 asserted edit groups to produce the standalone 816-line **StradFixed3.wl** and its unified diff.

The new mathematical capability is the higher-odd quadratic-field recipe. The principal engineering changes are conservative opaque admission, exact status semantics, incumbent preservation, more appropriate outer-candidate guards, context-aware positive caching with a recursion guard, corrected raw-argument validation, local resource wrappers, and budgeted cost snapshots. Certificate metadata is made more explicit without claiming an independent proof system.

8.2 Compatibility changes that deserve attention

The public context changes to **RadicalDenest3'**. Noncanonical algebraic **Root** presentations can be conservatively rejected. **NumericPrefilter** defaults to false and, when true, is diagnostic-only. Disabled reports can contain **Missing["NotComputed"]** costs. Malformed calls become **Failure** objects. The cost charges Gaussian components and stops opaque counting structurally, so some equal expressions may be ranked differently. New report metadata explains certificate scope and truncation. These are documented behavior changes, not drop-in compatibility guarantees.

Standalone helpers still have their own conservative bounded execution policy. They validate effective defaults before entering that policy, but are not an API for arbitrary held code. The main denester still leaves mixed-inexact trees and unsupported heads unchanged. The **Factor** option controls the generic **Factor** proposal; it does not promise to disable mathematical factorization throughout all algorithms.

8.3 Remaining limitations

A native Wolfram run is required before trusting evaluator details, printed forms, performance, message handling, or the complete regression suite. The revision still relies on the kernel for algebraic normalization and equality. It has no standalone proof checker, no complete multi-surd or general denesting decision procedure, no minimum-cost guarantee, and no hard process-level resource enforcement.

Positive memoization can still retain a merely adequate result. Whole-pass checkpointing can discard useful uncommitted work on interruption. The candidate pipeline explores one incumbent, not a full Pareto set of equivalent presentations. The source retains some low-priority cleanup opportunities, including unused locals/helpers and redundant candidate polishing; those were not removed merely to make the diff larger. A smaller patch is easier to compare with the previously tested implementation.

9 Verification and reproducibility

9.1 What actually ran

The independent script was executed with Python 3.13.5 and SymPy 1.14.0. It completed 14,190 assertions. These are *not 14,190 native package tests*. Most assertions concern exact polynomial identities, rational phase enumeration, and finite parameter families. The results are saved as both JSON and text.

Independent check group	Passed assertions
Direct quadratic norm, square, and branch checks	858
Indirect criterion and positivity checks	780
Cubic trace, norm, reconstruction, denominator identity	1,512
Higher odd-index identities and reconstruction	1,519
Horner inverse and Honsbeek polynomial/parameters	15
Kummer orbit coverage and rational-power phase membership	9,488
Multi-surd coset and unit-norm examples	6
Source hash, lexical balance, and patch-presence checks	12
Total	14,190

The direct family uses 286 signed cases. The indirect family contains 260 rational parameter cases. The cubic family has 378 rational quadratic-field reconstructions. The higher-odd sweep adds 504 reconstructions, plus seven symbolic Dickson identities. The branch calculations use rational angles modulo 2π for indices through sixteen. These checks provide real evidence for the formulas and enumeration logic, but do not establish the Wolfram implementation's behavior.

A lexical delimiter scan checks balanced brackets, nested comments, and strings in both packages and native test files. It is not a full Wolfram parser. The Git hash check establishes that the reviewed original is exact. The patch-presence checks establish that selected edits were applied; they are not semantic tests of those edits.

9.2 Native tests supplied but not executed

`tests/StradFixed3.wlt` contains 50 `VerificationTest` cases. They cover admission and rejection of numeric forms, exact equality, principal negative-root behavior, forced false numerical hints, option validation and defaults, opaque metrics, classical identities, hostile or throwing

proposal functions, assumptions isolation, conservative host policies, report costs, denominator inversion, incumbent preservation, residual-index admission, and the new odd-index recipe.

The search-state tests use deliberately mocked factor/subproblem generators. This makes a defect visible independently of whether another expensive algorithm happens to rediscover the lost result. Such tests should be supplemented by the original unified-B suite and randomized structured families on an actual kernel. Performance and timeout tests should record the kernel version, system, messages, resource flags, exact result, and whether verification itself completed.

The `evidence/native_execution_status.json` file records the absence of native execution. There is no fabricated pass report. Running the supplied native runner will create a separate report file; it must not be confused with the independent Python log.

9.3 Rebuild commands

From the root of the extracted archive:

```
python code/build_improved.py
python tests/verify_independent.py
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
wolframscript -file tests/run_tests.wls
```

The Python verification requires SymPy; the patch builder uses the standard library. The last command requires a separately available Wolfram installation and was not executed here. The PDF uses ordinary LaTeX packages and does not require downloaded fonts or external figure assets. File checksums and a source manifest accompany the archive.

10 Prioritized next steps

The first release gate should be native differential testing, not more heuristic breadth. Run the supplied API and state tests, then the complete unified-B battery, recording exact equality and cost changes separately from printed-form changes. Any failure of the stricter opaque admission policy should be classified as invalid input, unsupported valid representation, or timeout before relaxing the predicate.

The next architectural improvement should separate three data types: a proposal, an equality decision with its reason, and a committed rewrite record. This would remove ambiguity around numerical hints, search-limit exhaustion, cache entries, and certificate arrays. A small typed record can preserve the distinction between “not equal,” “not proved equal,” “not cheaper,” and “not searched.”

The scope of a future complete algorithm must specify its base field, permitted root indices, and treatment of roots of unity. Landau’s general theory distinguishes the roots-of-unity case from a general-base construction within one of optimal; it is not a blanket minimum-depth guarantee for this bounded implementation. The relevant representation may include a large splitting field, which is a different complexity measure from the size of a short input formula.[8, 6]

For mathematical breadth, a square-class representation with controlled scalar-coset exploration is more promising than simply adding visible primes to a nonlinear ansatz. For optimization quality, retain a bounded Pareto frontier of candidates and account for the host’s active maximum-depth branch. For performance, cache reusable exact field data and validated root descriptions, while retaining separate entries for search outcomes under different resources. Those changes should be measured independently; one large rewrite would make regressions difficult to localize.

11 Conclusion

StradFixed2.wl deserves credit for its exact-gate architecture and for correctly implementing several nontrivial denesting identities. The main remaining problems are not a mistaken quadratic formula or a missing roots-of-unity factor. They are mismatches between mathematical hypotheses and admitted representations, between heuristic information and exact status names, and between local proposal generation and global search state.

The proposed revision addresses those concrete boundaries and adds a proved higher-odd quadratic-field method. Its source is reproducibly derived from the exact reviewed blob, and its mathematical components have independent exact checks. It remains a proposed revision pending native execution. That distinction is essential: useful source proofs and algebraic verification support a better implementation, but do not replace running the actual language, kernel, and regression workload.

A Source map for future review

Original component	Pinned line range
Public contract, usages, and defaults	1–60
Session state and bounded operation wrapper	62–111
Option resolution	113–143
Admission grammar and cost	145–183
Equality and standalone helpers	185–221
Acceptance and polishing	223–250
Polynomial denominator inverse	252–278
Quadratic, cubic, Honsbeek, multi-surd recipes	280–395
Field factors and index reduction	397–461
Multiplier search	463–573
Generic and power proposals, memoization	575–640
Island recombination and host traversal	642–686
Session driver and public dispatch	688–745

B Minimal diagnostic probes

These examples are intended for a clean kernel with explicit package contexts. The second and third probes involve known source defects or controlled injections; none is reported here as an executed native transcript.

```
Get["code/StradFixed2.pinned.wl"];
Get["code/StradFixed3.wl"];

(* Exact representation is not algebraicity. *)
z = Root[#^5 + # - Pi &, 1];
{RadicalDenest2'ExactAlgebraicQ[z],
 RadicalDenest3'ExactAlgebraicQ[z]}
(* Expected policy difference: {True, False}. *)

(* A deliberately false numeric hint must not decide inequality. *)
Block[{RadicalDenest3'Private'numericallyDifferentQ = (True &)},
 RadicalDenest3'EqualityStatus[
  Sqrt[5 + 2 Sqrt[6]], Sqrt[2] + Sqrt[3]]]
(* Required result: "Equal". *)

(* Malformed public syntax must reach the validator. *)
```

```

RadicalDenest3'Strad[Sqrt[5 + 2 Sqrt[6]], 17]
(* Required result: Failure[...], not an unevaluated call. *)

(* A new trace--norm path, with the principal branch still gated. *)
RadicalDenest3'Strad[(239 + 169 Sqrt[2])^(1/7)]
(* Mathematically expected value: 1 + Sqrt[2]. *)

```

C References and source qualifications

References

- [1] RadicalDenest contributors. *StradFixed2.wl*. Pinned commit 9000d6f533a68ee7659a6830ad25bba401f800a5, 2026. <https://github.com/VladimirReshetnikov/RadicalDenest/blob/9000d6f533a68ee7659a6830ad25bba401f800a5/src/corrected/StradFixed2.wl>. Included byte-exact in this archive; repository license: MIT No Attribution.
- [2] RadicalDenest repository. *Unified analysis B*, `src/code-review/unified-B/unified_analysis_B.tex`, same pinned commit. <https://github.com/VladimirReshetnikov/RadicalDenest/tree/9000d6f533a68ee7659a6830ad25bba401f800a5/src/code-review/unified-B>. Prior design rationale and reported experiments; not independently reproduced here.
- [3] RadicalDenest repository. *Unified report on radical denesting*, `docs/report/radical_denesting_unified.tex`, same pinned commit. <https://github.com/VladimirReshetnikov/RadicalDenest/tree/9000d6f533a68ee7659a6830ad25bba401f800a5/docs/report>. Research synthesis and source guide.
- [4] RadicalDenest repository. *RETYPESET_ARTICLES.md*, `docs/literature`, same pinned commit. https://github.com/VladimirReshetnikov/RadicalDenest/blob/9000d6f533a68ee7659a6830ad25bba401f800a5/docs/literature/RETYPESET_ARTICLES.md. Editorial status of the six corrected re-typesettings.
- [5] RadicalDenest repository. *CORRECTIONS.md* for the BFHT re-typesetting, `docs/literature/papers/bfht-Decreasing-the-nesting-depth-of-expressions-involving-square-root/retype-set-2026/`. Same pinned commit. In particular, editorial items 10, 11, and 13 qualify recursive witness and completeness claims.
- [6] RadicalDenest repository. *landau1989.tex*, corrected re-typesetting of Landau's 1989 extended abstract, in `docs/literature/papers/landau92-Simplification-of-nested-radicals/retype-set-2026/`. Same pinned commit. This is not the 1992 journal paper and includes substantive editorial qualifications.
- [7] A. Borodin, R. Fagin, J. E. Hopcroft, and M. Tompa. *Decreasing the nesting depth of expressions involving square roots*. *Journal of Symbolic Computation* 1 (1985), 169–188. <https://www.cs.toronto.edu/~bor/Papers/decreasing-nesting-depth-for-expressions.pdf>. Original paper, including the positive-real scope of its higher-root classification.
- [8] S. Landau. *Simplification of nested radicals*. *SIAM Journal on Computing* 21 (1992), 85–110. DOI: 10.1137/0221009. The publisher's abstract distinguishes the roots-of-unity case from the general-base, within-one-of-optimal construction.
- [9] E. Gkioulekas. *On the denesting of nested square roots*. Author manuscript dated December 21, 2016; associated with the 2017 publication. <https://faculty.utrgv.edu/eleftherios.gkioulekas/papers/submitted/denesting-square-roots.pdf>. Direct and indirect formulas; the repository also supplies an edited 2026 re-typesetting.
- [10] D. J. Jeffrey and A. D. Rich. *Simplifying square roots of square roots by denesting*. Chapter 4 in M. J. Wester, ed., *Computer Algebra Systems: A Practical Guide*, Wiley, 1999. <https://www.uwo.ca/apmaths/faculty/jeffrey/pdfs/wester.pdf>.

- [11] A. Cavallo. *Denesting cubic radicals*. arXiv:2403.04776v2, September 28, 2024. <https://arxiv.org/html/2403.04776v2>. Used for comparison of the quadratic-field cubic criterion, not as a general complexity theorem.
- [12] M. Honsbeek. *Denesting certain nested radicals of depth two*. University of Nijmegen, Report 9916, 1999. Repository record: <https://repository.ubn.ru.nl/handle/2066/18722>. The external PDF could not be retrieved in this session; the algorithmic identity used here is explicitly derived and is also presented in the supplied repository report and unified-B review.
- [13] Wolfram Research. *Root* and *RootReduce*, Wolfram Language documentation, accessed September 5, 2026. <https://reference.wolfram.com/language/ref/Root.html>; <https://reference.wolfram.com/language/ref/RootReduce.html>. Parameterized and transcendental-coefficient roots; canonical algebraic representations.
- [14] Wolfram Research. *PossibleZeroQ*, Wolfram Language documentation, accessed September 5, 2026. <https://reference.wolfram.com/language/ref/PossibleZeroQ.html>. The exact-method guarantee is conditional on explicit algebraic inputs.
- [15] Wolfram Research. *Accuracy*, Wolfram Language documentation, accessed September 5, 2026. <https://reference.wolfram.com/language/ref/Accuracy.html>. Significance information is distinguished in this review from a separately proved enclosing interval.
- [16] Wolfram Research. *TimeConstrained*, Wolfram Language documentation, accessed September 5, 2026. <https://reference.wolfram.com/language/ref/TimeConstrained.html>. Interrupt granularity, abort protection, and process-accounting qualifications.